
東北大學
《人工智能范式下的软件需求分析与设计》配套课件
软件工程与 AI 范式变革
从经典软件工程方法论，到大模型驱动的生成式工程
第 1 章
范式认知 · 过程选型 · 项目立项
东北大学软件学院 · 软件工程专业
贯穿案例：企业级「智工单」智能工单路由与辅助分析系统
软件需求分析与设计
SRA&D · Courseware

先看一个问题

本章的切入点

1. 一个 30 人的团队，用最新的大模型把编码效率提升了 3 倍，为什么项目仍然延期 4 个月？
2. 一个需求只有 12 个字——「做个系统，把工单自动分给对的人」——它能直接变成代码吗？
3. 如果 AI 可以写出 80% 的代码，那么软件工程师剩下的 20% 是什么？这三个问题的答案，构成本章的全部内容。

本章在全书中位置

生命周期主线 × AI 赋能主线



后续章节依次为：架构设计（第 7 章）、详细设计（第 8 章）、专项设计（第 9 章）、综合实战（第 10 章）

贯穿案例：企业级「智工单」系统

贯穿案例 • 案例总览

业务背景

- 某中型制造企业 IT 服务台，日均接收员工报修工单 800+ 条
- 工单来源分散：邮件、电话、OA 表单、微信群，难以统一跟踪
- 人工分派错误率约 28%，平均首次响应时长 6.5 小时
- 重复性问题反复解答，知识沉淀在个人经验中

建设目标（四大 AI 能力）

- 工单自动分类路由：按文本内容分派到对应支持组
- 相似工单检索：基于语义相似度召回历史同类工单
- 解决方案智能推荐：从知识库中生成候选解决方案
- 工单量趋势预测：预测未来 7 天工单量并预警峰值

本课程的组织方式

全书 10 章共用这一个案例：每章前半部分讲授经典软件工程理论，后半部分讲解大模型赋能的实操方法，最后落地到「智工单」系统的对应阶段产出物，最终在综合实战中形成完整的一套需求与设计文档。

1.1 软件开发的本质与核心挑战

本节要解决的问题

为什么软件开发这么难？难在哪里？这种「难」是暂时的技术水平问题，还是软件这种人造物的固有属性？

1.1.1 软件的固有特性

- 复杂性 Complexity——软件是「人脑与人脑」之间最复杂的实体
- 不可见性 Invisibility——没有几何形状，无法用图纸完整表达
- 易变性 Changeability——软件是最容易被要求修改的工程制品

1.1.2 软件开发的经典困境

- 需求模糊、复杂度失控、变更难以管控
- 传统应对思路：抽象、模块化、分阶段开发、过程管控

软件的本质：Brooks 的三重论断

《没有银弹》1987 提出的经典判断

软件不是硬件，也不是数学。Frederick P. Brooks 在《人月神话》《没有银弹》中系统论证了软件的四个本质特征因而更需方法论支撑。

- **复杂性（Complexity）**
  - 软件实体由大量相互关联的构件组成，构件之间的依赖关系远多于硬件
  - 「规模放大」不会简单复制结构——10 万行代码不是 1 万行代码的 10 倍复杂
- **一致性（Conformity）**
  - 软件必须服从人为约定的接口、制度、法规模板，不能只服从自然规律
- **不可见性（Invisibility）**
  - 软件没有空间形状，无法像建筑图纸一样「看见」整体结构
- **易变性（Changeability）**
  - 软件被认为「容易改」，于是被持续要求修改；硬件一旦定型则很少变更

本节重点展开复杂性、不可见性、易变性三大特性，它们直接解释了后续所有工程方法的由来。

固有特性之一：复杂性

Complexity

复杂性不是「代码写得多」，而是「元素之间的关联关系多到超出人的认知负荷」。

复杂性的四个来源

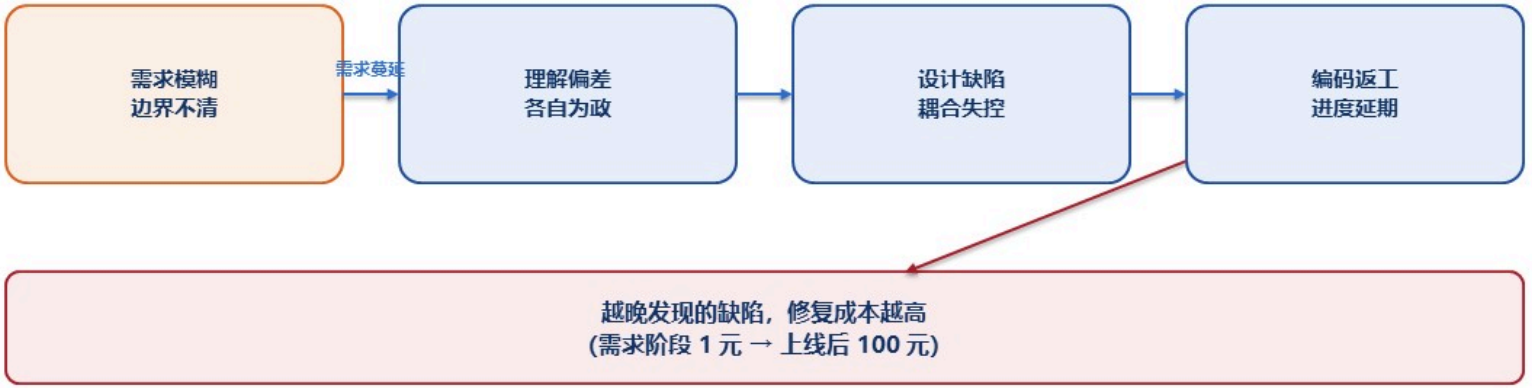
- 业务规则交织：一条「工单 SLA 规则」可能与分派规则、优先级规则、排班规则互相影响
- 状态空间爆炸：工单有 7 个状态、4 类优先级、3 类角色，组合状态可达数百种
- 接口耦合：智工单需对接 HR 系统、OA、邮件网关、知识库，任一接口变更都会传导
- 人员协作：30 人团队的沟通路径数量随人数平方增长

复杂性的工程含义

- **因为复杂，所以必须「分而治之」——这直接导致模块化、分层、抽象等设计手段的产生。**
- **因为复杂，所以必须「控制信息量」——这直接导致文档、模型、规格说明的必要性。**

复杂性如何演化为项目失败

从需求到缺陷的传导链



这就是「风险前置」原则的由来：在需求与设计阶段投入的每一分成本，都是在为下游十倍、百倍的返工成本买保险。

固有特性之二：不可见性

Invisibility

建筑物可以看图纸，机械可以看装配图，而软件运行时的结构在物理世界中没有对应物。

不可见性带来的三个具体问题

- 沟通困难：业务方看不懂代码，开发看不懂业务，缺少「共同可视物」
- 评审困难：无法像审图一样审代码，缺陷只能靠推演和测试暴露
- 认知负荷高：新人接手项目需要数周才能建立完整的「脑中地图」

工程应对：把不可见变成可见

- 用统一建模语言（UML）把结构画出来——类图、用例图、活动图就是软件的「工程图纸」
- 用分层与命名约定降低认知负荷——「高内聚低耦合」本质上是为降低理解成本服务

- **本课程第 4—9 章的核心工作，本质上就是持续生产这些「可视物」。**

固有特性之三：易变性

Changeability

在工程领域，软件是唯一被默认「可以改」的制品；而恰恰是这唯一可改的制品，改动成本最低的同时影响面最大。

易变性的来源

- 软件即服务：业务规则、监管要求、组织架构发生变化时，软件必须跟随
- 需求可见性效应：用户只有在看到可运行系统后，才能真正说清自己要什么
- 竞争压力：产品迭代节奏从「年」压缩到「周」

易变性对工程方法的硬性要求

- **必须区分「稳定内核」与「易变外围」——稳定部分做结构化设计，易变部分做可插拔设计**
- **必须建立变更控制机制——否则「易变」会退化为「范围蔓延」（Scope Creep）**
  - 本课程第 2 章的需求变更控制、第 6 章的设计模式，都是对易变性的直接回应

传统工程方法的四把「手术刀」

对应三大困境的经典应对思路

1 **抽象 Abstraction**

忽略与目标无关的细节，只保留本质特征。用「工单」这个概念覆盖所有类型的服务请求，从而把千变万化的业务统一为可处理的模型。

2 **模块化 Modularity**

把系统切成职责单一、边界清晰、可独立演化的单元。智工单划分为工单管理、智能分类、知识库、统计分析四大模块。

3 **分阶段开发 Phasing**

把一次巨大的不确定性拆成若干次可控的小不确定性，每阶段都有明确交付物与准出标准。

4 **过程管控 Control**

通过计划、评审、基线与变更控制，让「易变」不演化为「失控」，让进度与质量可度量、可追溯。

单选题 1分

Brooks 在《没有银弹》中指出，软件的本质困难（Essence）主要来自软件自身的固有特性。以下哪一项不属于他论述的软件固有特性?

A 复杂性（Complexity）

B 不可见性（Invisibility）

C 易变性（Changeability）

D 可移植性（Portability）

单选题 1分

某团队在项目上线后才暴露出「需求边界不清」的问题，修复成本极高。按照缺陷成本放大效应，最有效的应对策略是：

A 增加测试人力，在上线前尽可能多地暴露缺陷

B 在需求与设计阶段完成严谨的界定与评审，把缺陷拦截在上游

C 先快速上线，通过用户反馈逐步修正需求

D 引入更先进的自动化测试框架提高覆盖度

雨课堂 Rain Classroom

1.2 经典软件开发生命周期

本节知识地图

软件生命周期（SDLC）是把「从想法到运行」这一过程结构化的框架。本节按「线性 → 迭代 → 风险驱动 → 敏捷」的演进顺序展开。

1.2.1 瀑布模型与迭代模型

- 瀑布模型：阶段划分、线性执行、文档驱动——优点与局限
- 增量迭代模型：逐步交付、持续反馈——与瀑布的本质区别

1.2.2 螺旋模型与风险驱动开发

- 循环结构、每轮四象限活动、风险分析的核心地位
- 适合高风险、需求不明确项目的原因

1.2.3 敏捷开发与 Scrum 框架

- 敏捷宣言四价值观与 12 项原则
- Scrum 三大角色、三大工件、五大事件

瀑布模型：阶段划分与线性执行

Waterfall Model



瀑布模型的核心是「阶段化 + 文档驱动 + 里程碑评审」：上一阶段的交付物是下一阶段的唯一输入，每阶段结束前必须通过评审并基线化。

瀑布模型的适用场景与固有局限

优点与适用场景

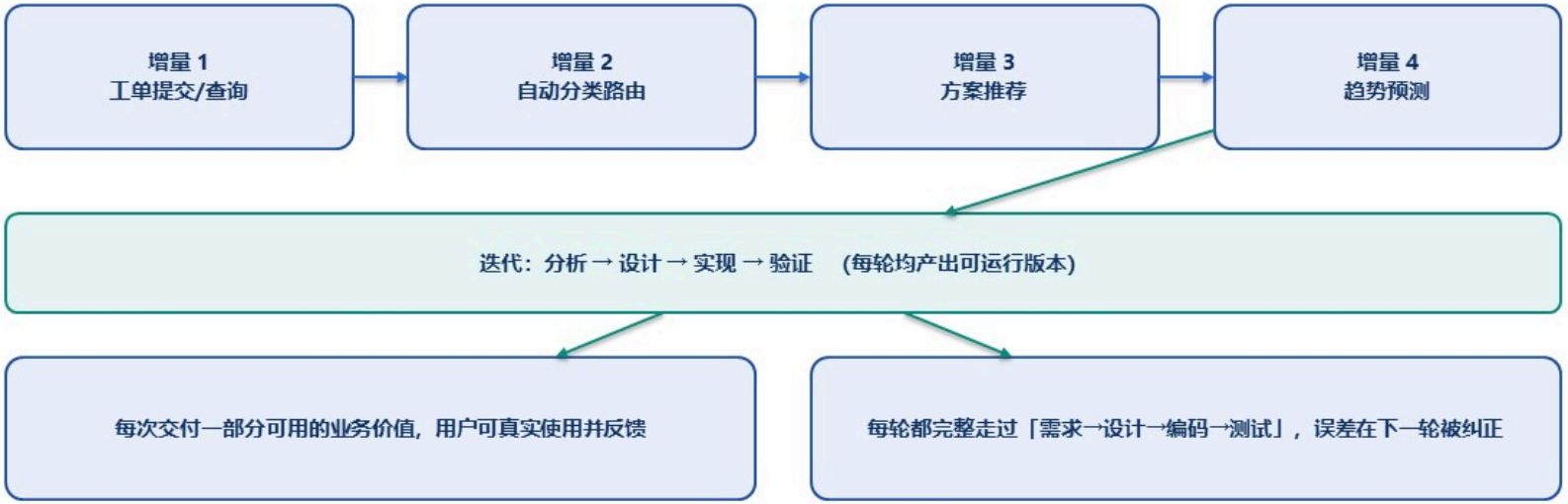
- 结构清晰、责任明确，每个阶段有确定的输入输出与评审点
- 文档完备，便于外包、审计与合规交付
- 进度与成本易于度量，合同化项目便于验收
- 适用于：需求高度明确且稳定、技术成熟、风险可控的项目
- 典型例子：银行核心系统改造、航天测控软件、法规遵从类系统

局限与失效场景

- 变更成本极高：需求变更需回溯到需求阶段重走全流程
- 反馈滞后：直到测试阶段用户才第一次看到可运行系统
- 风险后置：架构缺陷往往到集成阶段才暴露
- 「文档驱动」易异化为「文档表演」——文档齐全但无人阅读
- 不适用于：需求模糊、市场窗口短、需频繁试错的互联网产品

增量模型与迭代模型

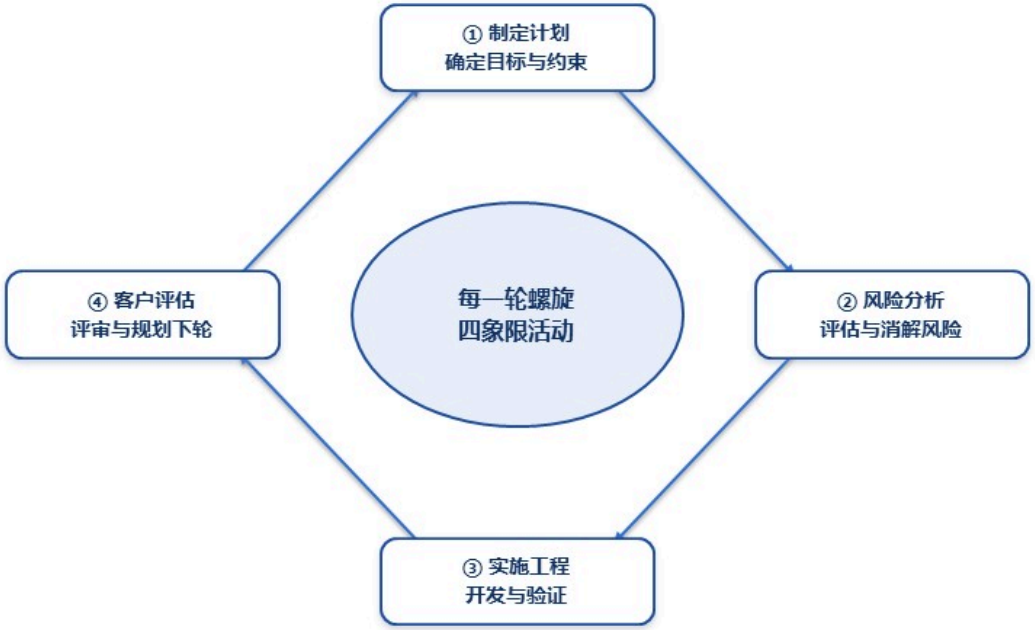
Incremental & Iterative



关键区别：增量是「分块交付功能」，迭代是「反复精化方案」。工程实践中两者通常结合使用，形成「增量迭代」。

螺旋模型：风险驱动的循环演进

Spiral Model · Barry Boehm 1988



螺旋模型 = 迭代模型 + 系统化风险分析。每一轮螺旋都从「目标设定」开始，以「风险分析」为决策中心，再进入工程实施，最后客户评估。

螺旋模型为什么适合高风险项目

风险驱动的决策逻辑

核心机制：把风险分析提升为独立且前置的必做活动

- 每轮开始时先问：本阶段最大的不确定性是什么？——技术可行性？数据可得性？模型精度？
- 针对最大风险设计「最小验证动作」：原型、技术试验、数据采样、概念验证（PoC）
- 风险未消解则不开工：宁可延长本轮，也不把风险带入下一轮

在「智工单」这类 AI 项目中的具体价值

- 第一轮螺旋：验证历史工单数据是否够用、标签质量是否达标（数据风险）
- 第二轮螺旋：验证文本分类模型能否达到 85% 准确率（技术风险）
- 第三轮螺旋：验证业务方是否接受「AI 建议 + 人工确认」的协作模式（组织风险）

- **AI 项目最大的风险通常不是「写不出代码」，而是「数据不可用 / 效果不达标 / 业务不采纳」。**

敏捷开发：四个价值观

Agile Manifesto · 2001

个体与互动

高于「流程和工具」

重视团队协作与沟通，而不是把人当作流程中的零件

可工作的软件

高于「详尽的文档」

交付可运行的软件才是价值，文档是手段不是目的

客户合作

高于「合同谈判」

与业务方建立持续协作关系，而不是一次性谈定需求

响应变化

高于「遵循计划」

计划是工具，随认知更新调整计划是能力而非失职

注意：敏捷宣言的表述是「左侧项目比右侧项目更有价值」，而不是「右侧项目没有价值」。文档、计划、合同、流程依然需要，只是不应压过人与响应变化。

Scrum 框架：三大角色

角色定义职责边界，避免「都负责等于没人负责」

产品负责人 PO

对产品价值负责，管理并排序产品待办列表（Product Backlog）

代表业务方做需求取舍与优先级决策

智工单案例：由 IT 服务台主管担任，决定「自动分类」与「方案推荐」哪个先做

敏捷教练 SM

对 Scrum 过程有效性负责，清除团队障碍

不分配任务、不做技术决策，是服务型领导

在 AI 项目中还需协调数据、算力、模型评审等跨部门阻塞项

开发团队 DT

跨职能自组织团队，对 Sprint 目标负责

AI 项目团队需包含：前后端、算法工程师、数据标注、测试

规模建议 3—9 人，智工单项目为 7 人

Scrum 框架：三大工件与五大事件

工件承载信息，事件产生节奏

类别	名称	核心内容与目的
工件	产品待办列表 Product Backlog	全部待实现需求的排序清单，由 PO 负责维护与优先级排序
工件	冲刺待办列表 Sprint Backlog	本 Sprint 选定的需求 + 实现计划，由开发团队承诺
工件	增量 Increment	本 Sprint 产出的、满足「完成定义」的可用产品增量
事件	冲刺 Sprint	固定时长的迭代容器（通常 2—4 周），其他事件都在其中发生
事件	冲刺计划会 Sprint Planning	确定 Sprint 目标与待办列表，回答「做什么」与「怎么做」
事件	每日站会 Daily Scrum	15 分钟同步进展与障碍，不解决问题本身
事件	冲刺评审会 Sprint Review	向利益相关者演示增量并收集反馈，更新待办列表
事件	冲刺回顾会 Retrospective	反思过程与协作方式，形成下一轮的改进项

单选题 1分

某团队要开发一套面向航天测控的地面软件，需求在立项阶段就已通过行业标准完全确定，且交付后需接受严格审计。最合适的过程模型是：

A 瀑布模型

B Scrum 敏捷开发

C 螺旋模型

D 极限编程 XP

雨课堂 Rain Classroom

单选题 1分

在 Scrum 框架中，负责对产品待办列表进行排序、并对产品价值负责的角色是：

A 敏捷教练（Scrum Master）

B 产品负责人（Product Owner）

C 开发团队（Developers）

D 项目经理（Project Manager）

雨课堂 Rain Classroom

单选题 1分

螺旋模型区别于一般迭代模型的最本质特征是：

A 采用更短的迭代周期

B 每个螺旋周期都把风险分析作为独立且前置的必做活动

C 强调文档的完整性

D 要求团队规模更小

雨课堂 Rain Classroom

1.3 需求工程与系统设计的核心价值

本节知识地图

前面讲的是「变化的过程」，本节回答：在软件生命周期中，哪一个环节对成败影响最大?

1.3.1 在软件工程中的核心地位与作用

- 目标锚定 · 风险前置 · 成本管控 · 协作桥梁

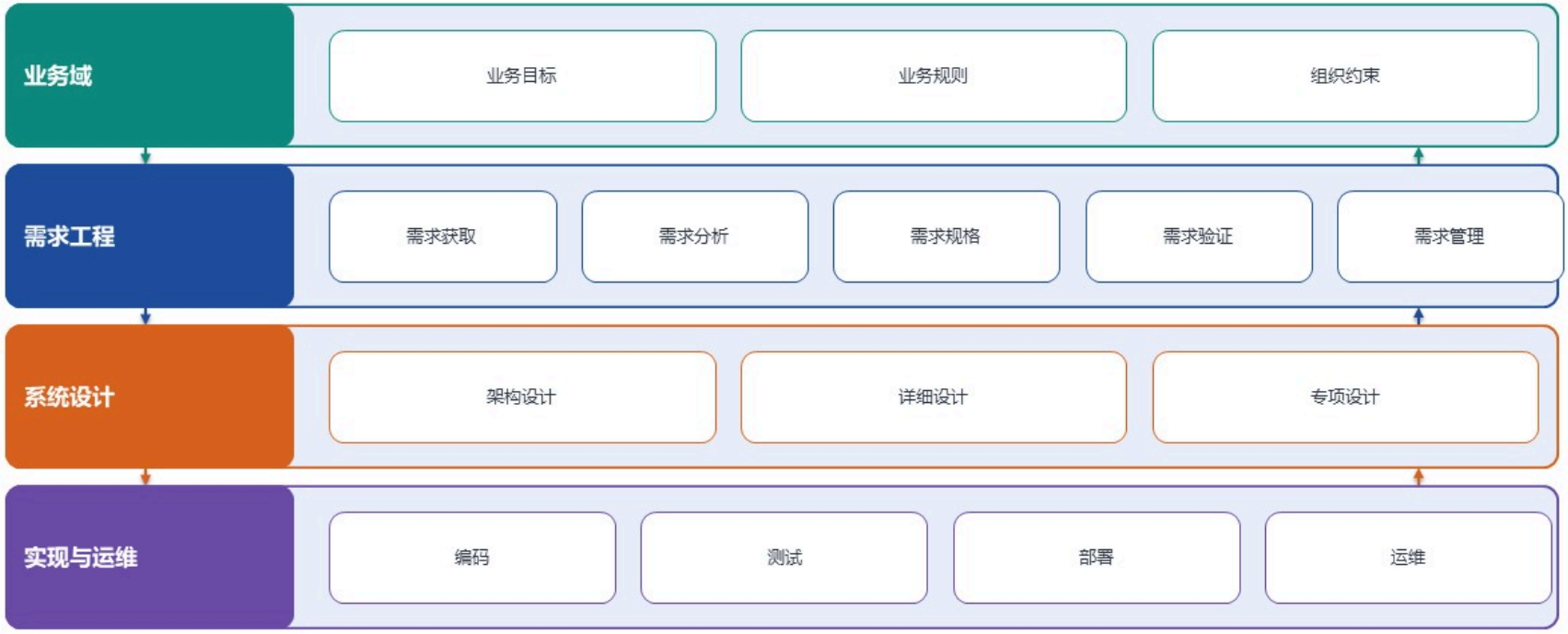
1.3.2 从需求分析到系统设计的演进逻辑

- 需求分析面向问题域，回答「做什么」
- 系统设计面向解域，回答「怎么做」

1.3.3 AI for Coding 时代的价值升维

- 上游质量决定 AI 产出上限；设计约束是 AI 编码的质量围栏
- 能力重心向上游迁移；风险前置的投入产出比显著提升

需求与设计在软件生命周期中的枢纽位置



需求工程与系统设计是「业务目标 → 技术实现」的转换枢纽：向上承接业务价值，向下决定架构与质量底座。

两个世界：需求分析 vs 系统设计

需求分析 —— 面向问题域

- 核心问题：系统应该为用户提供什么能力？（What）
- 关注对象：业务问题、用户诉求、外部行为、质量属性
- 主要产出：业务流程图、用例模型、领域概念模型、SRS
- 成功标准：完整、准确、无歧义、可验证
- 典型反例：「用 Redis 做缓存」不是需求，是设计决策

系统设计 —— 面向解域

- 核心问题：如何通过技术手段满足这些需求？（How）
- 关注对象：模块划分、接口定义、数据结构、技术选型
- 主要产出：架构图、类图与顺序图、数据库设计、接口规范
- 成功标准：可落地、可演进、满足约束与质量属性
- 典型反例：「系统要好用」不是设计，是尚未澄清的需求

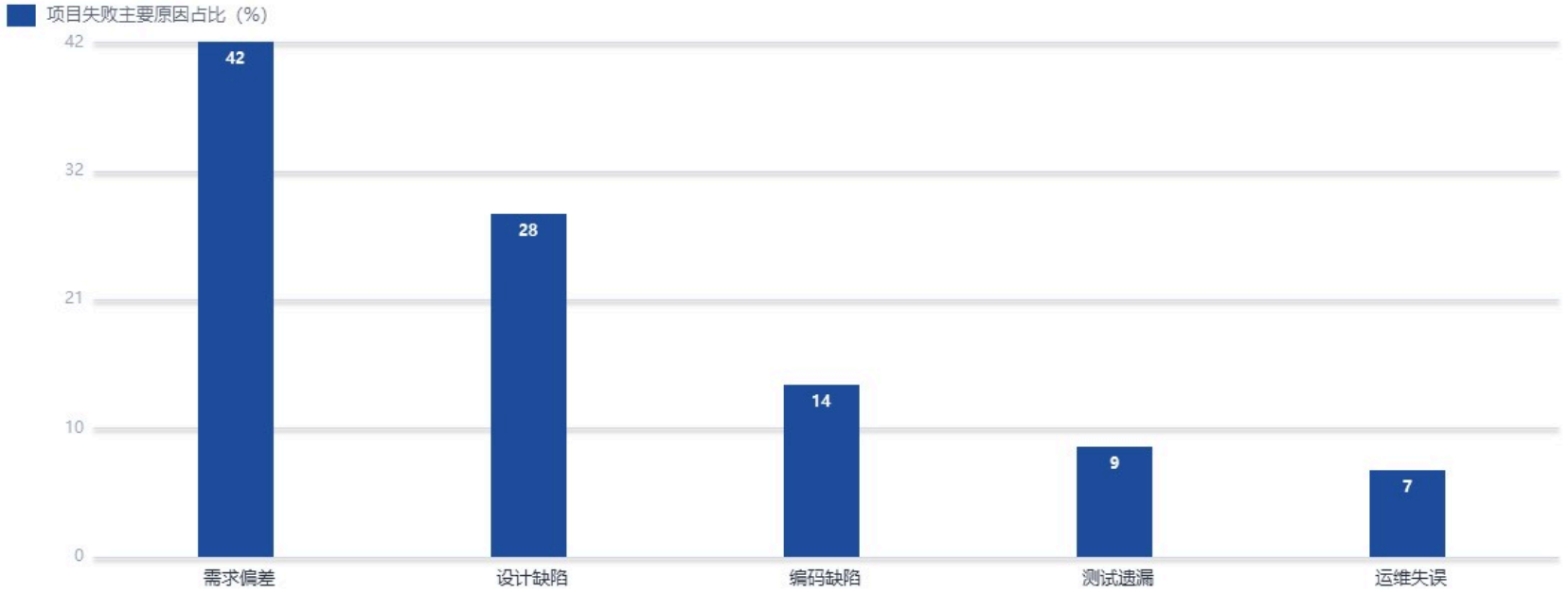
从需求分析到系统设计的演进逻辑



自顶向下逐层分解、自底向上逐层支撑：需求是设计的输入与依据，设计是需求的技术转化与落地保障，二者共同构成编码的前置依据。

为什么需求环节决定项目成败

行业数据的启示



约 70% 的软件项目失败可归因于需求与设计环节的偏差。需求与设计的投入占比不高，但对成败的影响权重最大。

AI for Coding 时代的价值升维（上）

当代码不再是瓶颈

大模型把编码效率提升了数倍，但软件项目的整体交付效率并未同比例提升——瓶颈向上游转移了。

① 上游质量决定 AI 产出上限

- 大模型生成代码的质量高度依赖输入的指令质量（需求 + 设计约束）
- 需求模糊时，AI 会生成大量「看似可用但不符合业务实际」的代码，带来更隐蔽的返工成本
- 精准完整的需求与清晰合理的设计，是 AI 生成高质量代码的前提与「施工图纸」

② 设计约束是 AI 编码的质量围栏

- AI 生成代码具有「局部最优、全局失控」的特点
- 缺少架构与接口约束时，零散生成的代码耦合度高、风格混乱、可维护性差
- 系统设计的价值在于为 AI 设定架构边界与规则约束，保障长期可演进性

AI for Coding 时代的价值升维（下）

能力重心的迁移

③ 能力重心向上游迁移，设计能力成为核心竞争力

- 常规编码工作被大幅替代后，开发者的核心价值从「实现功能」转向「定义问题、设计方案」
- 能否精准挖掘真实需求、设计合理架构、制定清晰技术方案，成为区分能力层级的核心标准

- **「会写代码」正在贬值，「会定义问题」正在升值。**

④ 风险前置的投入产出比显著提升

- AI 编码速度快、迭代成本低，但需求与设计缺陷会以更快速度传导到全部下游环节
- 错误一旦被 AI 规模化复制，后期修正代价被成倍放大
- 因此在需求与设计阶段投入精力把控质量，其投入产出比远高于传统开发模式

案例对照：同一个「自动分类」需求的两种写法

贯穿案例 · 对比演示

模糊写法（直接交给 AI）

- 「请写一个工单自动分类的功能」
- AI 输出：一个 Train/Predict 脚本，随机指定 5 个类别，无输入输出定义
- 结果：类别与真实支持组不一致；无置信度字段；无法与工单服务对接
- 隐患：代码「能跑」，但业务不可用，返工成本隐蔽且高昂

精确写法（先定义再生成）

- 输入：工单标题 + 描述（≤500 字），输出：类别 ID + 置信度 + Top-3
- 类别定义明确：网络组 / 桌面组 / 应用组 / 权限组 / 其他
- 约束：置信度 < 0.6 时返回「待人工分派」；响应时间 P95 ≤ 800ms
- 结果：AI 直接生成可对接的分类服务骨架与接口契约

结论

同一个模型，输入质量不同，产出价值相差一个数量级。需求与设计不是「AI 的替代对象」，而是「AI 效能的放大器」。